

Divide and Conquer, Part 2

Majority Element, Strassen's Algorithm

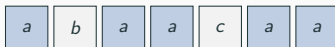
Finding the Majority

The Problem

Given $A[1 : n]$ whose elements can only be compared for equality.

Majority element

An element occurring strictly more than $n/2$ times in A .



a occurs $5 > 7/2$ times

Naive: count each element against the whole array, giving $O(n^2)$.

The Key Claim

Claim

Let L, R partition A . If A has a majority element, it is the majority of L or of R (or both).

Proof. Suppose z is the majority of A but of neither half. Then

$$\text{count}(z, L) + \text{count}(z, R) \leq \frac{|L|}{2} + \frac{|R|}{2} = \frac{n}{2},$$

contradicting that z occurs more than $n/2$ times in A . $\square$

So there are only **two candidates** to check.

A Divide-and-Conquer Algorithm

Recurse on both halves, then verify each candidate against the full array.

```
function majority(A)
    n = length(A)
    if n == 0
        return nothing
    elseif n == 1
        return A[1]
    end

    L = A[1:div(n,2)]; R = A[div(n,2)+1:end]
    x = majority(L); y = majority(R)

    count_x = sum(1 for a in A if a == x)
    count_y = sum(1 for a in A if a == y)

    count_x > n/2 && return x
    count_y > n/2 && return y
    return nothing
end
```

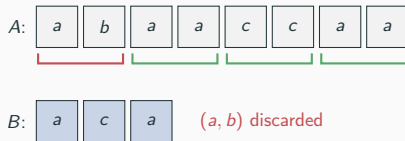
Two calls of half the size, plus $O(n)$ to count both candidates:

$$T(n) = 2 T(n/2) + O(n) = O(n \log n)$$

Better than $O(n^2)$. Can we reach $O(n)$?

A Better Idea: Pairing

Pair up neighbours. Keep **one copy** of a matching pair, **discard** a mismatched pair.



If m is the majority of A , then m is the majority of B .

The converse fails, so we must verify at the end.

Why the Majority Survives

Let m be the majority of A . Among the $n/2$ pairs, let

- x = pairs (m, m) , y = pairs with exactly one m , t = pairs with no m .

Why the Majority Survives

Let m be the majority of A . Among the $n/2$ pairs, let

- $x = \text{pairs } (m, m)$, $y = \text{pairs with exactly one } m$, $t = \text{pairs with no } m$.

Then $x + y + t = \frac{n}{2}$ and m occurs $2x + y > \frac{n}{2}$ times.

Why the Majority Survives

Let m be the majority of A . Among the $n/2$ pairs, let

- $x = \text{pairs } (m, m)$, $y = \text{pairs with exactly one } m$, $t = \text{pairs with no } m$.

Then $x + y + t = \frac{n}{2}$ and m occurs $2x + y > \frac{n}{2}$ times.

Subtracting the first from the second:

$$(2x + y) - (x + y + t) > 0 \implies x > t.$$

Why the Majority Survives

In the reduced array B :

- each (m, m) pair contributes exactly one $m \Rightarrow m$ appears x times;
- each pair with no m contributes *at most* one element.

So $|B| \leq x + t$, and since $x > t$,

$$2x > x + t \geq |B| \quad \Rightarrow \quad x > \frac{|B|}{2}.$$

m is still a majority element of B . $\square$

The Improved Algorithm

1. If n is odd, test $A[1]$ directly; if it is not the majority, drop it. $O(n)$
2. Pair up neighbours and build B as above. $O(n)$
3. Recurse: $m \leftarrow \text{majority}(B)$, where $|B| \leq n/2$.
4. Verify m against the **original** A . $O(n)$

$$T(n) = T(n/2) + O(n) = O(n)$$

Implementation

```
function majority_fast(A)
    n = length(A)
    n == 0 && return nothing
    n == 1 && return A[1]

    C = A # handle odd length
    if n % 2 == 1
        count = sum(1 for a in A if a == A[1])
        count > n/2 && return A[1]
        C = @view A[2:end]
    end

    B = Vector{eltype(A)}() # reduced array
    for i in 1:2:length(C)
        C[i] == C[i+1] && push!(B, C[i])
    end

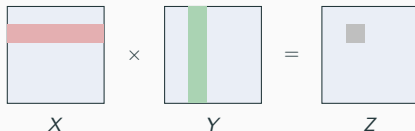
    x = majority_fast(B) # recurse, then verify
    if x !== nothing && sum(1 for a in A if a == x) > n/2
        return x
    end
end
```

Strassen's Algorithm

Matrix Multiplication

For $n \times n$ matrices X and Y , the product $Z = XY$ has

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}.$$



n^2 entries $\times O(n)$ each $= O(n^3)$ for the naive method.

Block Multiplication

Matrix multiplication works **blockwise** (assume n is a power of 2):

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \cdot \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

That is **8** multiplications of $(n/2) \times (n/2)$ blocks, plus $O(n^2)$ of additions:

$$T(n) = 8 T(n/2) + O(n^2) \stackrel{\text{Master}}{=} O(n^3).$$

No better than the naive algorithm.

Exercise

Verify block multiplication on this example, using blocks of size 2.

$$\begin{bmatrix} 1 & 2 & 3 & 2 \\ 5 & 2 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 2 & 3 & 5 & 7 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 1 & 2 & 1 \\ 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix} = \dots$$

Solution

With $A = \begin{bmatrix} 1 & 2 \\ 5 & 2 \end{bmatrix}$, $B = \begin{bmatrix} 3 & 2 \\ 1 & 0 \end{bmatrix}$, $E = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $G = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}$:

$$AE + BG = \begin{bmatrix} 3 & 2 \\ 7 & 2 \end{bmatrix} + \begin{bmatrix} 13 & 18 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 16 & 20 \\ 8 & 4 \end{bmatrix}.$$

Multiplying the full 4×4 matrices directly gives the same top-left block:

$$\begin{bmatrix} 16 & 20 & \cdot & \cdot \\ 8 & 4 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}$$

The other three blocks check out the same way.

Seven products instead of eight:

- $P_1 = A(F - H)$
- $P_2 = (A + B)H$
- $P_3 = (C + D)E$
- $P_4 = D(G - E)$
- $P_5 = (A + D)(E + H)$
- $P_6 = (B - D)(G + H)$
- $P_7 = (A - C)(E + F)$

Each sum or difference costs $O(n^2)$, so each P_i costs $T(n/2) + O(n^2)$.

Reassembling the Product

$$\begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

- $AE + BG = P_5 + P_4 - P_2 + P_6$
- $AF + BH = P_1 + P_2$
- $CE + DG = P_3 + P_4$
- $CF + DH = P_1 + P_5 - P_3 - P_7$

$$T(n) = 7 T(n/2) + O(n^2) = O(n^{\log_2 7}) \approx O(n^{2.81})$$

Implementation: Splitting into Blocks

Views avoid copying the quadrants out of A and B .

```
function strassen(A::AbstractMatrix, B::AbstractMatrix)
    n = size(A, 1)
    n <= 32 && return A * B # base case

    mid = div(n, 2)
    A11, A12 = @view(A[1:mid, 1:mid]), @view(A[1:mid, mid+1:end])
    A21, A22 = @view(A[mid+1:end, 1:mid]), @view(A[mid+1:end, mid+1:end])
    B11, B12 = @view(B[1:mid, 1:mid]), @view(B[1:mid, mid+1:end])
    B21, B22 = @view(B[mid+1:end, 1:mid]), @view(B[mid+1:end, mid+1:end])

    # continued on the next slide
```

Implementation: Products and Assembly

Seven recursive calls, then four block additions.

```
# ... still inside strassen(A, B)

P1 = strassen(A11, B12 - B22); P2 = strassen(A11 + A12, B22)
P3 = strassen(A21 + A22, B11); P4 = strassen(A22, B21 - B11)
P5 = strassen(A11 + A22, B11 + B22); P6 = strassen(A12 - A22, B21 + B22)
P7 = strassen(A11 - A21, B11 + B12)

C = Matrix{eltype(A)}(undef, n, n)
C[1:mid, 1:mid] .= P5 + P4 - P2 + P6; C[1:mid, mid+1:end] .= P1 + P2
C[mid+1:end, 1:mid] .= P3 + P4; C[mid+1:end, mid+1:end] .= P1 + P5 - P3 - P7
return C
end
```

- Large constant factors and numerical instability.
- Better algorithms exist for sparse matrices.
- Real implementations switch to the naive method below a threshold.

Still a beautiful example of what divide-and-conquer buys you.

Fast matrix multiplication is still active research,
the current best is $O(n^{2.3728596})$ (as of 2024).